

Who Explains What

Four of us, four different parts of the system. This brief tells you exactly which part is yours, what it does in plain English, and what to say when they ask.

23CSE498 Project Phase 2 Real-Time Adaptive Cross-Media Recommendation System brief cut 14 Sep

SPEAKS 1ST

Eswara Raj

Finding things — how a search becomes a shortlist

SPEAKS 2ND

Sanjay

Learning from behaviour — LightGCN and how we measure it

SPEAKS 3RD

Aditya

Reacting instantly — the live feedback loop

SPEAKS 4TH

Himaneesh

Deciding who may see what — age and domain filtering

Read this first

How the presentation works

Last review we each walked through the whole system from start to finish. Four people described the same thing four times, and it sounded repetitive. This time each person owns one piece and only that piece.

1. **Know three things about your part:** the formula, the one number, and the one weakness. If you can't say all three from memory, you're not ready yet.
2. **Only Eswara explains the overall architecture**, once, in about 30 seconds at the start. After that, nobody says “the pipeline” again.
3. **Every weak point is admitted before they find it.** Each section below has the exact words. Saying it first turns a problem into a design decision;

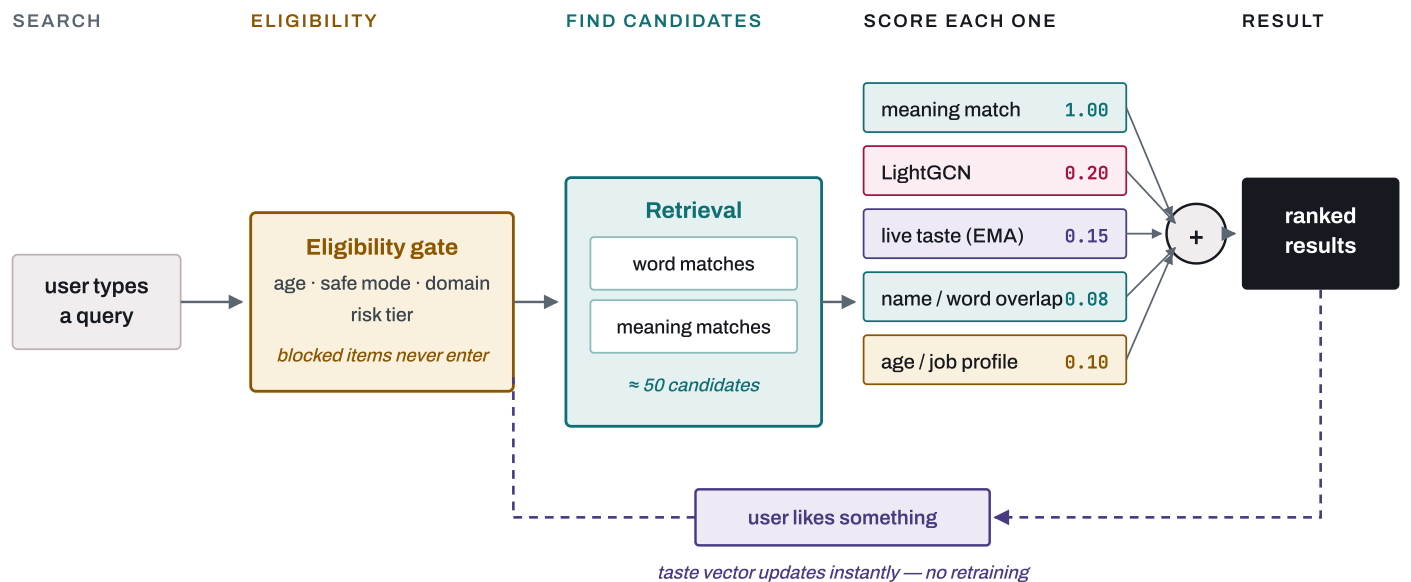
being caught turns it into a mistake.

4. Each weakness is mentioned once, by one person. If two of us apologise for the same thing, it looks like we never spoke to each other.
5. Hand over by naming the next person’s topic, not by saying “next.” For example: “...but matching words can’t tell us what people actually like. Sanjay.”

The big picture

The whole system in one diagram

Everything below is colour-coded to whoever owns it. Find your colour — that is your territory, and nothing outside it is your problem.



■ Eswara — retrieval & word matching ■ Sanjay — LightGCN & evaluation ■ Aditya — live feedback ■ Himaneesh — eligibility

How to read it: a search goes left to right. The eligibility gate runs *before* anything is found, so blocked items never even become candidates. Retrieval narrows 120,747 items down to roughly 50. Those 50 are each given five scores, which are added together with the weights shown. The dashed purple line is the feedback loop — liking something updates the user’s taste vector immediately.

Vocabulary

Words the panel may use

You don't need all of these — but if one comes up in a question, you shouldn't freeze. Learn the ones in your own section properly.

Embedding e_i	A piece of text turned into a list of 384 numbers, arranged so that things meaning similar things get similar lists. “Space film” and “sci-fi movie” land close together.
Cosine similarity $\cos(a, b)$	Measures whether two of those lists point in the same direction. 1 = same direction, 0 = unrelated. This is how we say “these two things are alike.”
Vector search / ANN	Searching by meaning instead of by keyword: encode the query, then find the closest item vectors. ANN = approximate nearest neighbour, a fast way to do it on 120,747 items.
Collaborative filtering	Recommending based on behaviour rather than content — “people who liked this also liked that.” It knows nothing about what the item <i>is</i> .
Implicit feedback	We never asked anyone to rate anything. We only know what they clicked, bought or liked — so we have positives but no reliable negatives.
Cold start	A brand-new item nobody has interacted with yet. Behaviour-based models can say nothing about it; meaning-based ones still can.
Recall@20	Of the things the user actually wanted, how many appeared in our top 20. Higher is better.
NDCG@20	Like Recall, but gives extra credit for putting the right thing near the <i>top</i> of the list, not just somewhere in it.
Baseline / control	The simple method we must beat to prove the complicated one is worth it. Ours is “just recommend the most popular items.”
Reranking	Retrieval fetches a shortlist; reranking re-sorts that shortlist using extra signals. Our five scores are rerankers.
Holdout / leave-last-out	To test fairly, we hide each user's most recent item, train on the rest, then check whether the model puts the hidden one back in the top 20.

Sparsity

Almost every user-item pair is empty. Our data is about 0.04% filled, which is why all the scores look small.

Finding things

Turning what someone typed into a shortlist of about 50 candidates, out of 120,747 items.

IN PLAIN ENGLISH

When someone searches, we have to narrow the whole catalogue down to a handful worth scoring. We do it two ways at once.

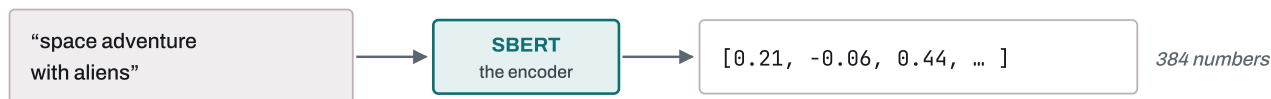
By meaning: every item's description is converted into a list of 384 numbers that captures what it's *about*. The search query gets converted the same way. Then we find the items whose numbers point in the most similar direction. This is why searching “space adventure with aliens” finds *Interstellar* even though those exact words appear nowhere in it.

By words: meaning-matching is bad at exact names. Ask for “James Cameron” and the meaning-based search returns things that *feel* like James Cameron films. So we also run a plain word-matching search in three tiers — person names first, then titles, then keywords — and put those on top.

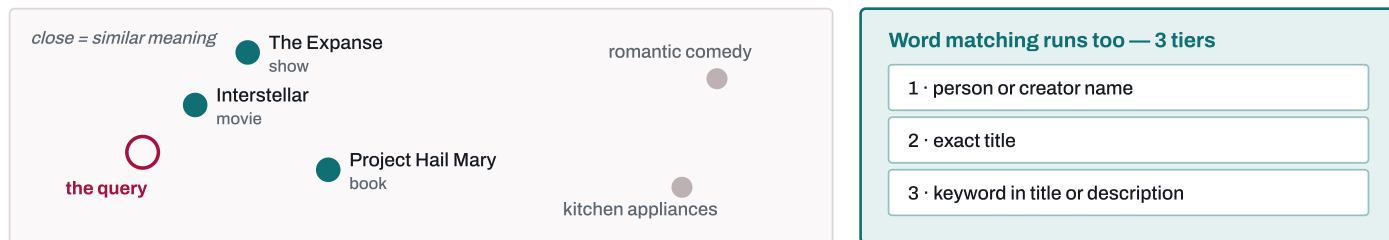
YOUR ONE-SENTENCE VERSION

“We search by meaning and by words at the same time, because each one fails where the other works.”

STEP 1 — TEXT BECOMES NUMBERS



STEP 2 — SIMILAR MEANINGS LAND NEAR EACH OTHER



Left: text becomes a point in space; things that mean similar things sit near each other, and the search finds the nearest points. **Right:** word matching runs alongside, because “James Cameron” needs an exact hit, not a vibe.

CATALOGUE	EACH ITEM IS	FORMULA
120,747	384	$\cos(e_i, q)$
items searched	numbers long	how aligned two items are

POINTS TO PREPARE

- The encoder is called **SBERT** (`all-MiniLM-L6-v2`). It produces 384 numbers per item. We chose a small one on purpose — it keeps the server cheap and fast.
- Queries, items *and* user taste vectors all live in the **same 384-number space**, so we can compare any of them directly. That’s why nothing needs converting between stages.
- We use a database called **Qdrant** for the search. The important bit: it can apply filters *during* the search, not after. The old option (FAISS) had to search first and filter afterwards — which would break Himaneesh’s guarantee.
- Word matches are capped at **half the page**, so one common keyword can’t flood the results.
- Every result records **which path found it** (word match or meaning match), and the UI shows it. Nice thing to point at live.
- The small **word-overlap score (0.08 weight)** exists for names. Our own tests show it scores 0.467 on “James Cameron” and 0.667 on “Taylor Swift”, but only 0.250 on a general topic query — it fires exactly when a name is involved.

- **Cold start:** meaning-based scores are the only ones that work for a brand-new item with no interaction history.

YOUR WEAKNESS — SAY IT YOURSELF

Matching meaning and words tells us what an item *is*, but nothing about what people actually *do*. Two films can be described almost identically and be loved by completely different audiences. That is exactly what the next part solves — hand over to Sanjay here.

IF THEY ASK ABOUT THE KNOWLEDGE GRAPH

SAY THIS, THEN STOP

“Today the knowledge-graph signal is word overlap between the query and the item’s title, description, creators and categories, weighted 0.08. The visualizer shows the target structure we’re building towards — turning it into a real graph database with path-based explanations is our Review-3 deliverable.”

Do not claim there is a live graph with nodes and edges — there isn’t one yet, and the deck now says so honestly. If they ask where “564 nodes” came from, that’s Sanjay’s to answer, not yours.

YOUR DEMO — ABOUT 30 SECONDS

Type one query. Show one result card. Point at the score breakdown bar and the badge showing which path found it. Then stop — personalisation belongs to Aditya.

QUESTIONS THEY MAY ASK

Why Qdrant rather than FAISS?

Because filters apply inside the search itself. FAISS would need a second lookup afterwards, which would let blocked items be retrieved and then removed — and that breaks the age guarantee.

Why such a small embedding model?

384 dimensions keeps the whole catalogue in memory and the server CPU-only. A bigger model buys accuracy we haven’t yet shown we need.

Isn't keyword search a step backwards?

It's a complement. Meaning-search is strong on intent and weak on exact names; keyword search is the opposite. Running both in tiers is what makes both kinds of query work.

Learning from behaviour

The part that learns from what people actually did, plus how we prove any of this works. The deepest section — and it owns both of our honest weak spots.

IN PLAIN ENGLISH

Imagine a huge web connecting every user to every item they liked. If you and I both liked the same five films, and you also liked a sixth, that sixth is probably worth showing me — even though nothing in its *description* says so.

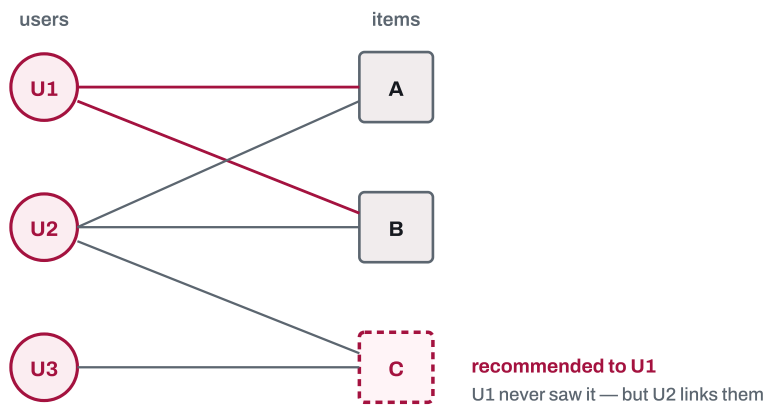
LightGCN learns that web. It gives every user and every item a list of 64 numbers, then repeatedly lets each one absorb information from its neighbours: users take on the character of items they liked, items take on the character of users who liked them. After three rounds, information has travelled three steps through the web — so it captures “people like you liked this” without anyone writing that rule.

We train it with a trick called **BPR**. Because we only know what people liked and never what they disliked, we can’t teach it to predict a rating. Instead we teach it comparisons: “this item the user liked should rank above this random one they didn’t touch.” Do that a few million times and the ordering becomes correct.

YOUR ONE-SENTENCE VERSION

“**LightGCN** learns what people’s behaviour has in common, which is something no amount of reading the description can tell you.”

USERS AND ITEMS FORM ONE WEB — INFORMATION SPREADS ALONG IT



Layer 1

U1 absorbs items it liked

Layer 2

...and the other users who liked those same items

Layer 3

...and what **those** users liked. Item C arrives here.

We stop at 3 — go further and everyone starts to look the same.

Why three layers: each layer lets information hop one more step through the web.

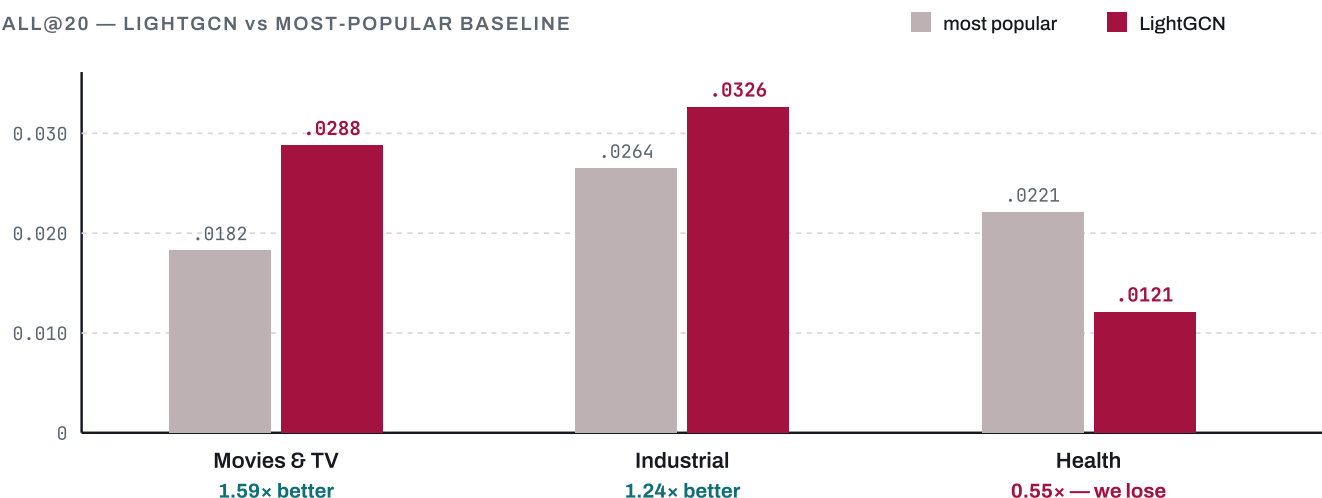
Three is enough to reach useful strangers; beyond that everyone's numbers blur together and the model stops telling people apart. That blurring has a name — over-smoothing.

NUMBERS PER USER/ITEM	LAYERS	TRAINED ON
64 embedding size d	3 hops through the web	Kaggle T4 60 epochs, GPU

HOW WE PROVE IT WORKS — AND WHERE IT DOESN'T

We tested on **Amazon Reviews 2023**, a real public dataset, in three different domains. For each one we hide each user's most recent purchase, train on everything else, then check whether the model puts the hidden item back in the top 20. We compare against the simplest possible method: **just recommend whatever is most popular**.

RECALL@20 — LIGHTGCN vs MOST-POPULAR BASELINE



Two wins and one loss, shown honestly. On health, simply recommending best-sellers beats our model. Don't hide this — the explanation below is stronger than the result.

THE HEALTH RESULT — REHEARSE THIS ANSWER

SAY THIS BEFORE ANYONE ASKS

“LightGCN loses on health, and that is the result. Household goods are commodity purchases — nearly everyone buys the same top sellers, so popularity is a genuinely strong baseline there. It tells us collaborative filtering helps where taste varies between people, and helps less where it doesn’t.”

THE ONE DEEP POINT THAT WILL IMPRESS THEM

If the conversation goes technical, this is your best material — and we found it by reading our own code.

- We **train** the model to compare items using a plain multiplication of the two number-lists (a dot product).
- But when we **use** the model, we first shrink every list to the same length before comparing (normalising), which turns it into a cosine.
- Shrinking the *user* side changes nothing about the order of results. Shrinking the *item* side does — because in this kind of model, an item’s “length” quietly encodes how popular it is.
- So normalising throws away the popularity knowledge the model learned. **And health is the one domain where popularity matters most.** That may be the whole explanation for the 0.55× result.
- But it’s a genuine trade-off, not a bug: that same normalising is what puts all five scores on the same scale, which is what makes the weights (1.00, 0.20, 0.15, 0.08, 0.10) mean anything at all.

Status to state clearly: it’s a scoped experiment, not a finished one. We know exactly what to run and we haven’t run it yet.

YOUR SECOND WEAKNESS — SAY IT BEFORE THEY OPEN THE FILE

The trained model file we ship with the running app is a small synthetic test set — 13 users, 347 items. Say: *“The implementation is complete and tested. The file currently loaded is a synthetic smoke-test, because our own platform has no real user base yet. That is exactly why we benchmark on Amazon Reviews 2023 instead.”*

This caveat is **yours alone**. The knowledge-graph visualizer is built from the same file, so it is one caveat, not two — Eswara will not repeat it.

ONE DETAIL THAT LOOKS LIKE A MISTAKE BUT ISN'T

In our results files, `hit_rate` and `recall` are *identical*. That's correct, not copy-paste: we hide exactly one item per user, so “did we find it” and “what fraction did we find” are the same question. You can verify live: $0.02884 \div 20 = 0.0014422$, which is exactly the precision we report.

PROTECT THE SMALL-LOOKING NUMBERS

All the scores look tiny. Explain why before they judge: the data is about **0.04% filled** — almost every user-item pair is empty. Published results on this kind of data sit in the 0.03–0.08 range. **The claim we're testing is “LightGCN beats popularity”, not “the number is large.”**

YOUR DEMO

The benchmark table and the Kaggle notebook. No live UI — that's Aditya and Himaneesh.

QUESTIONS THEY MAY ASK

Why LightGCN and not a normal neural network?

In this problem there are no input features to transform — the numbers *are* the thing being learned. The original paper showed that removing the transformation layers and activation functions actually improves results, because they only add capacity to overfit.

Why not train it to predict ratings?

We only have positives — we know what people bought or liked, never what they rejected. So we train on comparisons instead of values, which is what BPR does.

Finance has no result. Why?

It's the newest domain and the smallest — 3,103 items. The pipeline is identical and only one constant changes, so it's queued, not blocked. It's listed as Review-3 work.

Reacting instantly

The part that makes the word “real-time” in our project title true. You get the best live demo of the review — take your time with it.

IN PLAIN ENGLISH

LightGCN is powerful but slow to update — retraining it takes hours. So if someone likes three science-fiction films right now, it can’t react today.

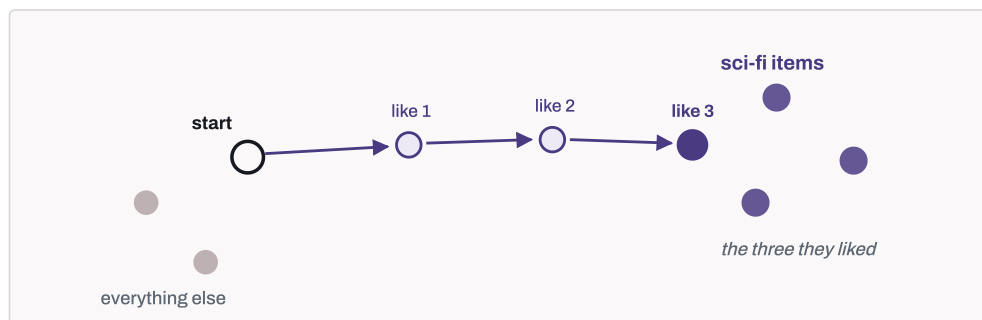
Our fix is deliberately simple. We keep one “taste vector” per user — the same 384 numbers everything else uses. Every time they like something, we nudge that vector a little towards the thing they liked. Every time they skip something, we nudge it away.

The nudge is 25% of the way ($\alpha = 0.25$), so roughly the last four things you did shape what you see. It’s one write to the database — no model is retrained, no item vectors move, nothing is rebuilt. That’s why the results change between one search and the next.

YOUR ONE-SENTENCE VERSION

“Retraining the graph model takes hours; this takes one write — so the system reacts inside a single session.”

THE TASTE VECTOR MOVES A LITTLE WITH EVERY LIKE



The whole formula

$$\text{new} = 0.25 \times \text{item} + 0.75 \times \text{old}$$

That's it. One line.

No training, no rebuild,
nothing else moves.

Each like moves the vector 25% of the way toward that item. After three likes it sits firmly in the sci-fi neighbourhood, so the next search ranks sci-fi higher — without anything being retrained.

NUDGE SIZE $\alpha = 0.25$ 25% toward the new item	COST PER EVENT 1 write vs hours for a retrain	WEIGHT IN BLEND 0.15 of the final score
--	--	--

POINTS TO PREPARE

- The taste vector lives in the **same 384-number space** as items and queries, so comparing them needs no conversion.
- It updates **the moment the event is logged**, not on a schedule. Positive events: like, bookmark, complete, high rating. Negative: skip.
- **Why not just retrain the model a bit each time?** A retrain updates the model's parameters and moves item vectors, which would disturb results for everyone. One user's vector move affects only that user.
- Storage is **MySQL**: one table of interactions and one of user profiles holding the vector. There's an index on (user_id, timestamp) because reading a user's recent history is the hot path.
- There's a **foreign key** from interactions to the catalogue, checked before insert — so a bad item ID returns a clean “400 bad request” instead of crashing with a 500.
- If the store is unavailable, **this signal switches off and the other four still rank**. Nothing breaks.

YOUR WEAKNESS — SAY IT YOURSELF

There's no session boundary and no time decay yet, so one accidental click drifts the vector permanently. Session-scoped vectors and a decay term are Review-3 work.

YOUR DEMO — REHEARSE UNTIL IT'S AUTOMATIC

1. Run a search. Read out the top two or three results.
2. Like three sci-fi items.
3. Run **the exact same search again**.
4. Point at the EMA segment now appearing in the score bar, and the ranking having moved.
5. Show the new row in the MySQL interactions table.

THE LINE THAT LANDS IT

“Nothing was retrained between those two searches.”

QUESTIONS THEY MAY ASK

Why 0.25 specifically?

It gives roughly a four-interaction memory — fast enough to feel immediate, damped enough that one click doesn't erase an established profile. It's configurable.

Isn't this just a recency trick?

Yes, deliberately. It covers the time window LightGCN can't: the graph model learns long-term patterns over hours of training, this one covers the current session. Different time scales, not competing methods.

What if a user likes something by accident?

Right now it sticks, which is the honest limitation above. A decay term fixes it and is planned.

Deciding who may see what

This is what the faculty asked for on 9 September — age filtering and domains beyond entertainment. It goes last because it's the strongest finish.

IN PLAIN ENGLISH — AND READ THE NEXT LINE CAREFULLY

This is not a model. The other three explained things that learn from data. This one is a set of rules that decides what a viewer is allowed to see at all. Nothing here is trained; it's about 550 lines of ordinary logic.

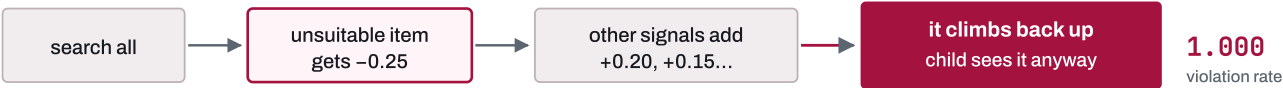
The important design choice: we check eligibility **before** searching, not after. The age and domain rules are built into the database query itself. So an item a child shouldn't see is never even fetched — it can't be found, can't be scored, and no other part of the system can push it back up.

We used to do it the other way: unsuitable items got a score penalty that pushed them down. That failed, because the four scoring signals could simply push them back up again. Moving the check *before* the search is what turned it from “usually works” into “cannot fail.”

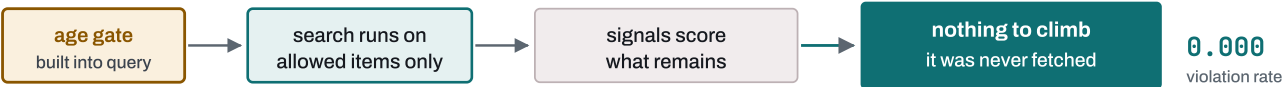
YOUR OPENING SENTENCE — USE IT EXACTLY

“The other three presented scoring models. This is not a model — it's a filtering stage, and the fact that it *can't* be a model is the point.”

THE OLD WAY — PENALISE AFTERWARDS · FAILS



THE NEW WAY — FILTER BEFORE SEARCHING · CANNOT FAIL



The line to land:
“Zero isn’t a good score — it’s a guarantee. There is nothing for the other signals to promote.”

Same rules, two places to apply them, completely different outcome. This one picture carries your whole section — the panel will remember it.

VIOLATION RATE	WITHOUT IT	LEARNED PARAMETERS
0.000	1.000	0
with the filter on	every result unsuitable	it’s rules, not a model

WHY IT MUST BE RULES AND NOT A MODEL — YOUR CORE ARGUMENT

SAY THIS

“A trained safety classifier is wrong sometimes by definition — you can measure the error rate, tune it, and it will still let things through in ways you can’t predict. A rule compiled into the query is wrong zero times by construction. For age-appropriateness we want the guarantee, not the accuracy score.”

WHAT WE MEASURED

VIEWER	AGE USED	CATALOGUE THEY CAN SEE	FILTER ON	FILTER OFF
Child	8	4.1% (4,987)	0.000	1.000
Teen	15	54.5% (65,824)	0.000	1.000
Adult	25	100% (120,747)	0.000	0.000
Adult, safe mode	7	4.1% (4,987)	0.000	1.000
No age given	13	54.5% (65,824)	0.000	1.000

THE RULES — EVERY ONE OF THEM FAILS SAFE

- **No age given is not treated as an adult.** It caps at 13. Guessing high would be the dangerous default.
- **Safe mode caps below a real adult age**, so an adult can deliberately ask for family-appropriate results.
- **An item with missing metadata** falls back to its content type's default, never to "unrestricted."
- **Risk tiers:** 0 = ordinary, 1 = show an advisory, 2 = needs a licensed professional and is withheld by default.
- **Health and finance are product discovery only** — never diagnosis, treatment, or investment advice. Say this sentence out loud; it's the responsible-scoping answer they're listening for.

DOMAINS — THE GENERALITY CLAIM

- Everything domain-specific lives in one config file. The search engine itself only ever sees text and a type label.
- **Adding a new vertical is a config entry, not new code.** TV shows were added exactly that way — that's your proof, not a promise.
- Current spread: entertainment 77,644 · health 20,000 · industry 20,000 · finance 3,103.

THE ONE SCORE YOU DO OWN

The age/job profile signal (weight 0.10) is yours — simple keyword matching with fixed weights. It's the **only hand-set, non-learned score of the five**, which is a neat contrast to draw against Sanjay's section.

YOUR WEAKNESS — SAY IT YOURSELF

These are **keyword rules, not certified age ratings**. Where a real rating already exists on an item we keep it; everything else is derived from keywords. Proper certified ratings and an audit log of blocked items are Review-3 work.

YOUR DEMO — THE FINISH

Run the same search at **age 8** and **age 25**, side by side. Then switch domains. Point out that the domain list is fetched live from the server — so a vertical added in the config file appears in the interface with no front-end change at all.

QUESTIONS THEY MAY ASK

Aren't you just hiding content?

Yes — that's the intended behaviour, and it's measurable. A child sees 4.1% of the catalogue, an adult sees 100%. We report the cost as well as the protection.

How do you know the age tags are right?

We don't claim they're certified. Where a real rating exists we keep it; otherwise it's a keyword heuristic that can only restrict, never relax. That direction matters — errors fail safe.

Could you give medical advice with this?

No, and that's deliberate. Health and finance are scoped to product discovery, an advisory is shown, and anything requiring a licensed professional is withheld by default.

Before tomorrow

Checklist

☐ **Everyone: learn your three things**

Your formula, your one number, your one weakness — from memory, no notes.

☐ **Everyone: rehearse your weakness sentence out loud**

These are written word-for-word above. Said confidently they sound like judgement; read hesitantly they sound like excuses.

☐ **Two full run-throughs together, timed**

Especially the handovers. That's where the repetition crept in last time.

☐ **Sanjay: fix the finance number in the deck**

It says 725 items; the catalogue now has 3,103.

☐ **Sanjay: commit the working tree**

Config, four test files, six front-end files and the catalogue builder are all modified but uncommitted. Do this before anyone demos from another laptop.

☐ **Aditya: run the like-then-search demo three times**

It's the best live moment we have and the one most likely to go wrong under pressure.

☐ **Himaneesh: check the age 8 / age 25 demo works end to end**

Including the domain switcher pulling its list from the server.

If they ask what this project is

The one-line answer

A domain-agnostic recommendation engine — pluggable domains, five ranking signals, a live feedback loop, and a hard eligibility layer — shown working across four different verticals and measured against a popularity baseline on three public datasets.

Not “a film recommender that also does health.” **The fact that it generalises is the contribution.**

Prep brief · 14 September 2026 · figures verified against the repository

Catalogue 120,747 items × 20 columns · embeddings 120,747 × 384 · 207 tests across 11 suites · 8 API endpoints · 19 front-end components

Benchmarks from reports/benchmarks/ · eligibility table from reports/constraint_evaluation.md